

Lý thuyết chuyên sâu: Số học Modulo (Modular Arithmetic)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Phép chia có dư & Quan hệ đồng dư thức (Congruence)

1.1. Phép chia có dư (Division Algorithm)

Với hai số nguyên a và b ($b \neq 0$), luôn tồn tại duy nhất một cặp số nguyên q (thương - quotient) và r (số dư - remainder) thỏa mãn:

$$a = b \cdot q + r \quad \text{với } 0 \leq r < |b|$$

Ta ký hiệu phép lấy số dư là:

$$r = a \bmod b$$

Nếu $r = 0$, ta nói a chia hết cho b (ký hiệu $a : b$) hay b là ước của a (ký hiệu $b \mid a$).

1.2. Đồng dư thức (Congruence Relation)

Cho số nguyên dương $m \geq 1$ (gọi là *modulo*). Hai số nguyên a và b được gọi là **đồng dư với nhau theo modulo m** nếu hiệu của chúng chia hết cho m :

$$a \equiv b \pmod{m} \iff m \mid (a - b)$$

Nói cách khác, a và b có cùng số dư khi chia cho m :

$$a \equiv b \pmod{m} \iff (a \bmod m) = (b \bmod m)$$

1.3. Các tính chất đại số cơ bản của đồng dư thức

Đồng dư thức là một quan hệ tương đương (phản xạ, đối xứng, bắc cầu) và bảo toàn qua các phép toán số học:

- Phản xạ:** $a \equiv a \pmod{m}$.
- Đối xứng:** $a \equiv b \pmod{m} \iff b \equiv a \pmod{m}$.

3. **Bắc cầu:** Nếu $a \equiv b \pmod{m}$ và $b \equiv c \pmod{m}$ thì $a \equiv c \pmod{m}$.

4. **Cộng/Trừ theo vế:**

$$a \equiv b \pmod{m} \text{ và } c \equiv d \pmod{m} \implies a \pm c \equiv b \pm d \pmod{m}$$

5. **Nhân theo vế:**

$$a \equiv b \pmod{m} \text{ và } c \equiv d \pmod{m} \implies a \cdot c \equiv b \cdot d \pmod{m}$$

6. **Lũy thừa với số mũ tự nhiên $k \in \mathbb{N}$:**

$$a \equiv b \pmod{m} \implies a^k \equiv b^k \pmod{m}$$

7. **Nhân cả hai vế với một hằng số:**

$$a \equiv b \pmod{m} \implies c \cdot a \equiv c \cdot b \pmod{m}$$

8. **Nhân cả hai vế và modulo với số nguyên dương $k > 0$:**

$$a \equiv b \pmod{m} \iff k \cdot a \equiv k \cdot b \pmod{k \cdot m}$$

9. **Chia cả hai vế cho ước chung:**

- Nếu $a \cdot c \equiv b \cdot c \pmod{m}$ và $\gcd(c, m) = 1$ thì:

$$a \equiv b \pmod{m}$$

- Trường hợp tổng quát với $d = \gcd(c, m)$:

$$a \cdot c \equiv b \cdot c \pmod{m} \iff a \equiv b \pmod{\frac{m}{d}}$$

10. **Lấy BCNN của các modulo:**

$$a \equiv b \pmod{m_1} \text{ và } a \equiv b \pmod{m_2} \iff a \equiv b \pmod{\text{lcm}(m_1, m_2)}$$

2. Các phép toán cơ bản theo Modulo

2.1. Phép cộng, trừ, nhân

Từ các tính chất trên, khi thực hiện nhiều phép tính số học ta có thể lấy modulo ở từng bước trung gian:

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$$

$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

Lưu ý số âm trong C++: Trong ngôn ngữ C/C++, toán tử `%` là phép chia lấy phần dư theo định nghĩa đại số của máy tính, kết quả của `-5 % 3` là `-2`. Để thu được phần dư chuẩn thuộc đoạn $[0, m - 1]$, ta luôn dùng công thức:

```
cpp int mod(int a, int m) { return (a % m + m) % m; }
```

2.2. Phép lũy thừa nhị phân (Binary Exponentiation)

Tính $a^b \bmod m$ với b lên tới 10^{18} trong thời gian $O(\log b)$:

Dựa trên tính chất chia để trị:

$$a^b = \begin{cases} 1 & \text{nếu } b = 0 \\ (a^{b/2})^2 \bmod m & \text{nếu } b \text{ chẵn} \\ (a \cdot a^{b-1}) \bmod m & \text{nếu } b \text{ lẻ} \end{cases}$$

3. Nghịch đảo Modulo (Modular Multiplicative Inverse)

3.1. Định nghĩa

Cho hai số nguyên a và m . Số nguyên x được gọi là **nghịch đảo modulo m** của a (ký hiệu a^{-1} hoặc $a^{-1} \pmod{m}$) nếu:

$$a \cdot x \equiv 1 \pmod{m}$$

Định lý về sự tồn tại:

Nghịch đảo modulo m của a **tồn tại khi và chỉ khi** a và m nguyên tố cùng nhau, tức là:

$$\gcd(a, m) = 1$$

Khi tồn tại, nghiệm $x \in [0, m - 1]$ là duy nhất.

3.2. Tính nghịch đảo khi modulo m là số nguyên tố (Định lý Fermat nhỏ)

Định lý Fermat nhỏ (Fermat's Little Theorem):

Nếu p là số nguyên tố và a không chia hết cho p ($\gcd(a, p) = 1$), thì:

$$a^{p-1} \equiv 1 \pmod{p}$$

Nhân cả hai vế với a^{-1} :

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

Do đó, ta tính nghịch đảo modulo p nguyên tố bằng hàm lũy thừa nhị phân:

$$a^{-1} = \text{pow_mod}(a, p-2, p)$$

Độ phức tạp: $O(\log p)$.

3.3. Tính nghịch đảo khi modulo m bất kỳ: Thuật toán Euclid mở rộng

Khi m không nguyên tố nhưng $\gcd(a, m) = 1$, ta áp dụng phương trình Bezout:

$$a \cdot x + m \cdot y = \gcd(a, m) = 1$$

Lấy modulo m hai vế:

$$a \cdot x \equiv 1 \pmod{m}$$

Vậy nghiệm x tìm được từ thuật toán Euclid mở rộng chính là nghịch đảo modulo m của a .

Thuật toán Euclid mở rộng:

$$\begin{aligned} \gcd(a, b) &= \gcd(b, a \bmod b) \\ a \cdot x + b \cdot y &= b \cdot x_1 + (a \bmod b) \cdot y_1 \\ &= b \cdot x_1 + (a - \lfloor a/b \rfloor \cdot b) \cdot y_1 \\ &= a \cdot y_1 + b \cdot (x_1 - \lfloor a/b \rfloor \cdot y_1) \end{aligned}$$

Từ đó:

$$x = y_1, \quad y = x_1 - \lfloor a/b \rfloor \cdot y_1$$

Độ phức tạp: $O(\log(\min(a, m)))$.

3.4. Định lý Euler và Phi hàm Euler (Euler's Totient Function)

Tổng quát hóa định lý Fermat nhỏ cho modulo m hợp số bất kỳ:

- **Hàm phi Euler** $\varphi(m)$: Số lượng số nguyên dương $k \leq m$ thỏa mãn $\gcd(k, m) = 1$.

- Nếu $m = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k}$, thì:

$$\varphi(m) = m \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right)$$

- **Định lý Euler:** Với mọi a nguyên thỏa mãn $\gcd(a, m) = 1$:

$$a^{\varphi(m)} \equiv 1 \pmod{m} \implies a^{-1} \equiv a^{\varphi(m)-1} \pmod{m}$$

3.5. Tiền xử lý nghịch đảo modulo từ 1 đến N trong $O(N)$

Trong các bài toán tổ hợp, ta cần tính $i^{-1} \pmod{p}$ cho mọi $1 \leq i \leq N$ với p nguyên tố.

Nếu dùng Fermat nhỏ từng số, tổng thời gian là $O(N \log p)$. Ta có thể tối ưu về $O(N)$ như sau:

Xét phép chia có dư của p cho i :

$$p = k \cdot i + r \implies k = \lfloor p/i \rfloor, \quad r = p \bmod i$$

Theo đồng dư:

$$k \cdot i + r \equiv 0 \pmod{p} \iff r \equiv -k \cdot i \pmod{p}$$

Nhân cả hai vế với $i^{-1} \cdot r^{-1}$:

$$i^{-1} \equiv -k \cdot r^{-1} \equiv -\lfloor p/i \rfloor \cdot (p \bmod i)^{-1} \pmod{p}$$

Chuyển về số dư dương:

$$i^{-1} \equiv (p - \lfloor p/i \rfloor) \cdot (p \bmod i)^{-1} \pmod{p}$$

Vì $p \bmod i < i$, giá trị $(p \bmod i)^{-1}$ đã được tính trước đó trong mảng!

3.6. Tiền xử lý nghịch đảo giai thừa $(k!)^{-1} \pmod{p}$ với $k = 1 \dots N$ trong $O(N)$

Để tính nhanh hệ số nhị thức $\binom{n}{k} = \frac{n!}{k!(n-k)!} \pmod{p}$ với số lượng truy vấn lớn ($Q \leq 10^6$), ta cần mảng nghịch đảo giai thừa $\text{invFact}[k] = (k!)^{-1} \pmod{p}$.

Phương pháp 1: Lan truyền lùi từ $N!$ (Backward Propagation - Chuẩn CP tối ưu nhất)

- **Cơ sở toán học:**

Ta có đẳng thức giai thừa:

$$k! = (k-1)! \cdot k$$

Lấy nghịch đảo modulo p hai vế:

$$(k!)^{-1} \equiv ((k-1)!)^{-1} \cdot k^{-1} \pmod{p}$$

Nhân hai vế với k :

$$((k-1)!)^{-1} \equiv (k!)^{-1} \cdot k \pmod{p}$$

- **Thuật toán thực thi:**
- Tính mảng giai thừa $\text{fact}[k] = k! \bmod p$ từ $k = 1 \dots N$.
- Tính duy nhất **một lần** nghịch đảo modulo của $N!$ bằng định lý Fermat nhỏ:

$$\text{invFact}[N] = (\text{fact}[N])^{p-2} \bmod p$$

- Lặp lùi dần từ $k = N$ về 1:

$$\text{invFact}[k-1] = (\text{invFact}[k] \cdot k) \bmod p$$

- **Độ phức tạp:**
- Thời gian: $O(N + \log p)$ (chỉ gọi `power` đúng 1 lần duy nhất cho $N!$).
- Bộ nhớ: $O(N)$ cho 2 mảng `fact` và `invFact`.

Phương pháp 2: Nhân dồn nghịch đảo từng phần tử (Forward Prefix Product)

- Tính mảng nghịch đảo các số nguyên $i^{-1} \pmod{p}$ cho $i = 1 \dots N$ theo mục 3.5.
- Tính mảng nghịch đảo giai thừa xuôi từ $1 \dots N$:

$$\text{invFact}[0] = 1$$

$$\text{invFact}[k] = (\text{invFact}[k-1] \cdot k^{-1}) \bmod p$$

- **So sánh 2 phương pháp:**
- *Phương pháp 1 (Lùi):* Chỉ tốn 1 lần tính nghịch đảo, không cần mảng `inv[i]`, tiết kiệm bộ nhớ, tốc độ chạy nhanh hơn, là chuẩn mực được dùng phổ biến nhất trong CP.
- *Phương pháp 2 (Xuôi):* Cần thêm mảng `inv[i]`, nhưng hữu ích nếu bài toán vừa yêu cầu chia cho $k!$ vừa yêu cầu chia cho các số đơn lẻ k .

4. Phép chia Modulo tổng quát khi $\gcd(b, m) > 1$

Nếu đề bài yêu cầu tính $\frac{a}{b} \bmod m$ với giả thiết $b \mid a$, nhưng $\gcd(b, m) = g > 1$ (nghĩa là b không có nghịch đảo modulo m):

Áp dụng tính chất nhân cả hai vế và modulo:

$$\frac{a}{b} \bmod m = \frac{a \bmod (b \cdot m)}{b}$$

Chứng minh:

Giả sử $a \bmod (b \cdot m) = r \implies a = q \cdot (b \cdot m) + r$.

Vì $b \mid a$ và $b \mid (b \cdot m) \implies b \mid r$.

Chia cả hai vế cho b :

$$\frac{a}{b} = q \cdot m + \frac{r}{b} \implies \frac{a}{b} \equiv \frac{r}{b} \pmod{m}$$

Vì $0 \leq r < b \cdot m \implies 0 \leq \frac{r}{b} < m$, nên $\frac{r}{b}$ chính là số dư duy nhất khi chia $\frac{a}{b}$ cho m .

5. Định lý Thặng dư Trung Hoa (Chinese Remainder Theorem - CRT)

5.1. Dạng cơ bản (Các modulo đôi một nguyên tố cùng nhau)

Cho hệ phương trình đồng dư:

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \vdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

Nếu $m_1, m_2, \dots, m_k$ nguyên tố cùng nhau từng đôi một ($\gcd(m_i, m_j) = 1$ với mọi $i \neq j$), hệ luôn có nghiệm duy nhất theo modulo $M = m_1 \cdot m_2 \cdot \dots \cdot m_k$:

$$x = \left(\sum_{i=1}^k a_i \cdot M_i \cdot M_i^{-1} \right) \bmod M$$

Trong đó:

- $M = \prod_{i=1}^k m_i$

- $M_i = \frac{M}{m_i}$

- M_i^{-1} là nghịch đảo modulo m_i của M_i (tồn tại vì $\gcd(M_i, m_i) = 1$).

5.2. Dạng tổng quát (Extended CRT - Modulo không nguyên tố cùng nhau)

Xét hệ gồm hai phương trình bất kỳ:

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \end{cases}$$

Biến đổi thành phương trình nghiệm nguyên:

$$x = m_1 \cdot p + a_1 = m_2 \cdot q + a_2 \iff m_1 \cdot p - m_2 \cdot q = a_2 - a_1$$

Đặt $g = \gcd(m_1, m_2)$.

- **Điều kiện có nghiệm:** Phương trình có nghiệm khi và chỉ khi:

$$(a_2 - a_1) \vdots g$$

Nếu không chia hết, hệ **vô nghiệm**.

- Nếu chia hết, giải phương trình Diophantus tìm p bằng Euclid mở rộng.

Nghiệm mới gộp lại thành:

$$x \equiv x_0 \pmod{\text{lcm}(m_1, m_2)}$$

Bằng cách lặp lại việc gộp 2 phương trình thành 1 phương trình mới sau $k - 1$ bước, ta giải được hệ k phương trình bất kỳ.

6. Các định lý nâng cao bổ trợ

6.1. Định lý Lucas (Lucas' Theorem)

Dùng để tính tổ hợp chập k của n modulo số nguyên tố p nhỏ ($p \leq 10^5$, trong khi $n, k \leq 10^{18}$):

Biểu diễn n và k trong hệ cơ số p :

$$n = n_m p^m + n_{m-1} p^{m-1} + \dots + n_0$$

$$k = k_m p^m + k_{m-1} p^{m-1} + \dots + k_0$$

Khi đó:

$$\binom{n}{k} \equiv \prod_{i=0}^m \binom{n_i}{k_i} \pmod{p}$$

(Quy ước $\binom{n_i}{k_i} = 0$ nếu $n_i < k_i$).

6.2. Thuật toán Baby-step Giant-step (Thuật toán Shanks)

Dùng để giải bài toán **Logarit rời rạc**: Tìm số nguyên dương x nhỏ nhất thỏa mãn:

$$a^x \equiv b \pmod{m}$$

Độ phức tạp $O(\sqrt{m})$ bằng phương pháp Meet-in-the-middle (chia căn).

7. Tài liệu tham khảo & Bài giảng đề xuất (References)

7.1. Giáo trình & Sách thuật toán kinh điển

1. **Competitive Programmer's Handbook** — *Antti Laaksonen*:
2. [Chapter 21: Number Theory \(Modular Arithmetic, Modular Inverse, Chinese Remainder Theorem\)](#).
3. **Introduction to Algorithms (CLRS 4th Edition)** — *Thomas H. Cormen et al.*:
4. *Chapter 31: Number-Theoretic Algorithms* — Cơ sở lý thuyết về giải thuật Euclid mở rộng, phương trình đồng dư tuyến tính và Định lý Thặng dư Trung Hoa.
5. **Competitive Programming 4 (CP4 - Book 1)** — *Steven Halim, Felix Halim, Suhendry Effendy*:
6. *Section 5.3: Number Theory & Modular Arithmetic*.
7. **Elementary Number Theory (7th Edition)** — *David M. Burton*:
8. *Chapter 4: Theory of Congruences* — Các chứng minh giải tích toán học chặt chẽ về hệ thặng dư đầy đủ và hàm số học.
9. **Tài liệu Chuyên Tin (Tập 2)** — *Thầy Hồ Sĩ Đàm (Chủ biên)*:
10. *Chương: Một số kiến thức Số học ứng dụng trong Tin học*.

7.2. Bài giảng & Chuyên đề trực tuyến (Online Resources & Wikis)

1. **VNOI Wiki**:
2. [Modulo cơ bản](#) — Bài viết nền tảng về đồng dư thức và phép toán số học.
3. [Nghịch đảo Modulo \(Modular Inverse\)](#) — Thuật toán Fermat nhỏ và nghịch đảo tuyến tính $O(N)$.
4. [Thuật toán Euclid & Euclid mở rộng](#) — Giải phương trình Diophantus.
5. [Phi hàm Euler & Định lý Euler](#) — Mở rộng cho modulo hợp số.
6. **CP-Algorithms (Algorithms for Competitive Programming)**:
7. [Modular Inverse](#) — Đầy đủ các phương pháp tính nghịch đảo.
8. [Chinese Remainder Theorem](#) — Cài đặt CRT và Garner's Algorithm.
9. [Linear Congruence Equation](#) — Giải phương trình $ax \equiv b \pmod{m}$.
10. [Binary Exponentiation](#) — Lũy thừa nhị phân và ứng dụng nhân ma trận.
11. **MIT OpenCourseWare (6.042J - Mathematics for Computer Science)**:
12. [Lecture 7 & 8: Number Theory and Modular Arithmetic](#) — Bài giảng toán học rời rạc đại học MIT.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.